

Explicit bounds for the product-kernel block

Claude, with Daniel Murfet

2026-09-07

Abstract

For a nonnegative kernel g the product-kernel block $J(t) = \iint_{(0,b]^2} u^{h_1} v^{h_2} g(tu^{k_1} v^{k_2}) du dv$ is bounded explicitly rather than asymptotically (Astra #5(d)). With equal exponents p and $R = b^{k_1+k_2}$ the exact density is $(k_1 k_2)^{-1} r^{p-1} \log(R/r)$, and for $t \geq 1$

$$J(t) \leq \frac{t^{-p}}{k_1 k_2} (\log t \cdot M_p(g) + U_{p,R}(g)), \quad M_p = \int_0^\infty s^{p-1} g, \quad U_{p,R} = \int_0^\infty s^{p-1} \log_+(R/s) g;$$

for bounded $0 \leq g \leq G_0$, $J(t) \leq G_0 b^{h_1+1} b^{h_2+1} / ((h_1+1)(h_2+1))$ for every t . These constants replace the `IsBig0` mixed-remainder bound and are what the strict-gap parameter wrapper (unit 99) needs as an integrable $K(w)$. Zero sorry/axiom.

1 The things formalised

```
noncomputable def prodKernel (b : ℝ) (h h k k : ℕ) (g : ℝ → ℝ) (t : ℝ) : ℝ :=
  u in Ioc (0 : ℝ) b, u ^ h * v in Ioc (0 : ℝ) b, v ^ h * g (t * (u ^ k * v ^ k))
theorem prodKernel_eq_twoDamp ... :
  prodKernel b h h k k g t = twoDamp 0 b t h h k k (fun _ _ s => g s)
noncomputable def kernelMoment ( : ℝ) (g : ℝ → ℝ) : ℝ :=
  s in Ioi (0 : ℝ), s ^ ( - 1) * g s
noncomputable def kernelLogMoment (R : ℝ) (g : ℝ → ℝ) : ℝ :=
  s in Ioi (0 : ℝ), s ^ ( - 1) * logPlus (R / s) * g s
theorem integral_inv_Icc (b : ℝ) (h : 0 < ℝ) (hb : ℝ b) :
  u in Icc b, u ^ 1 = Real.log b - Real.log
theorem log_density_factor ... :
  Real.log b - Real.log ((r / b ^ k) ^ ((k : ℝ) ^ 1))
  = 1 / (k : ℝ) * (Real.log (b ^ (k + k)) - Real.log r)
theorem prodKernel_eq_density (b p : ℝ) (h h k k : ℕ) (hb : 0 < b) (hk : 0 < k)
  (hk : 0 < k) (hp : ((h : ℝ) + 1) / k = p) (hp : ((h : ℝ) + 1) / k = p)
  (g : ℝ → ℝ) (hg : Continuous g) (t : ℝ) (ht : 0 < t) :
  prodKernel b h h k k g t
  = r in Ioc (0 : ℝ) (b ^ (k + k)),
    1 / ((k : ℝ) * k) * r ^ (p - 1) * (Real.log (b ^ (k + k)) - Real.log r)
    * g (t * r)
theorem prodKernel_le_equal ... (hg0 : ℝ s, 0 < g s)
  (hM : IntegrableOn (fun s => s ^ (p - 1) * g s) (Ioi 0))
  (hU : IntegrableOn (fun s => s ^ (p - 1) * logPlus (b ^ (k + k) / s) * g s) (Ioi 0))
  (t : ℝ) (ht : 1 < t) :
  prodKernel b h h k k g t
  t ^ (-p) / ((k : ℝ) * k)
  * (Real.log t * kernelMoment p g + kernelLogMoment p (b ^ (k + k)) g)
```

```

theorem prodKernel_le_of_bounded (b G : ) (h h k k : ) (hb : 0 < b) (g : → )
  (hg : Continuous g) (hgG : s, g s G) (t : ) :
  prodKernel b h h k k g t
    G * (b ^ (h + 1) / ((h : ) + 1)) * (b ^ (h + 1) / ((h : ) + 1))

```

2 Mathematics

$J(t)$ is the block integral of unit 86 with $\beta = 0$ and the constant amplitude $g(s)$, so the one-variable density is $k_2^{-1}r^{p-1}g(tr)\int_\varepsilon^b u^{-1}du$ with $\varepsilon = (r/b^{k_2})^{1/k_1}$, and $\log b - \log \varepsilon = k_1^{-1}\log(R/r)$. Substituting $x = tr$ (`integral_Ioc_rpow_mul_comp_mul_left`), $J(t) = \frac{t^{-p}}{k_1 k_2} \int_0^{tR} x^{p-1}(\log t + \log(R/x))g(x) dx$, and on $(0, tR]$ the bracket lies between 0 and $\log t + \log_+(R/x)$ (lower bound from $x \leq tR$); extending the integral to $(0, \infty)$ over the nonnegative dominating integrand gives the bound. The bounded kernel estimate is Fubini for the product of the two monomial integrals on the product box.

3 Paper-facing meaning

The mixed remainder of the rectangular decomposition is dominated by $J(t)$ with $g(s) = e^{-\beta s^2/2 + \beta L s}(1+s)^D$ -type kernels and shifted exponents; the explicit bound makes the remainder term of the $d = 2$ cutoff theorem quantitative, which is the prerequisite for instantiating the strict-gap parameter wrapper on mixed critical/noncritical blocks (the constants must be integrable in the parameter w , which pointwise `IsBig0` does not give). The unequal-exponent and gap-uniform bounds of Astra #5(d) are the natural follow-up.

4 Lean notes

- `setIntegral_congr_fun` on the density hands the amplitude as an unreduced `(fun x s => g s) u (t * r)`; `beta_reduce` before rewriting with `integral_mul_const`.
- Lower bounds for $\log(R/x)$ on $(0, tR]$: go through `Real.log_le_log hx0 hx.2` and `Real.log_mul`, not through inverses and divisions.
- Continuity of a parametric integral in the outer variable is expensive to prove directly; integrating over the product box (`twoDAmp_eq_prod`, `integral_mono_ae`, `integral_prod_mul`) avoids it entirely.